

LAPORAN WORKSHOP DEVOPS
BAB 3: DOCKER NETWORK, VOLUME, BIND MOUNT, TMPFS, DAN DOCKER
COMPOSE



Oleh:

Zhulva Priya Abhipraya (3126640034)

Dosen Pengajar:

Dr. Ferry Astika Saputra, ST, M.Sc

PROGRAM STUDI SARJANA TERAPAN LANJUT JENJANG TEKNIK
INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

1. PENDAHULUAN DAN TUJUAN PRAKTIKUM

1.1 Latar Belakang dan Landasan Teori

Dalam pengoperasian aplikasi berbasis kontainerisasi, kontainer tidak dapat dipandang sebagai entitas terisolasi yang berdiri sendiri. Arsitektur modern membutuhkan mekanisme interkomunikasi antarkontainer yang aman serta pengelolaan persistensi data yang andal. Secara teoritis, jaringan kontainer bertindak sebagai graf keterjangkauan (reachability graph) yang mendefinisikan batas-batas komunikasi antar-layanan. Misalnya, jaringan frontend menghubungkan reverse proxy ke aplikasi, sedangkan jaringan backend menghubungkan aplikasi ke database. Layanan basis data tidak perlu bergabung ke jaringan frontend maupun memublikasikan port ke host publik, sehingga memperkecil attack surface pada sistem.

Di sisi penyimpanan, kontainer bersifat ephemeral (replaceable). Writable layer kontainer mengikuti umur lifecycle kontainer, sehingga data bisnis yang membutuhkan persistensi harus dipisahkan ke mekanisme storage khusus. Docker menyediakan tiga opsi utama mount: Named Volume (dikelola oleh Docker Engine di `/var/lib/docker/volumes` untuk data persisten seperti database), Bind Mount (memetakan direktori host secara langsung untuk kebutuhan live development dan konfigurasi), serta tmpfs (menyimpan data sementara di RAM host untuk kebutuhan cache atau data sensitif yang tidak boleh tersimpan di disk).

Untuk mengoordinasikan seluruh komponen tersebut, Docker Compose hadir sebagai spesifikasi deklaratif berbasis YAML. Dengan Compose, infrastruktur multi-kontainer yang mencakup services, networks, volumes, environment variables, hingga mekanisme healthcheck dapat didefinisikan dalam satu unit versi (version-controlled artifact). Healthcheck memastikan ketergantungan antar-layanan terpenuhi secara fungsional (seperti memastikan PostgreSQL siap menerima koneksi sebelum aplikasi dinyalakan) menggunakan atribut `condition: service_healthy`.

1.2 Tujuan Pembelajaran dan Kompetensi Utama

Setelah menyelesaikan seluruh rangkaian praktikum Bab 3 ini, mahasiswa diharapkan mampu:

- Membuat dan mengonfigurasi user-defined bridge network serta membuktikan fitur otomatis DNS name resolution antar-kontainer.
- Menganalisis perbedaan karakteristik, peruntukan, dan tingkat keamanan antara Named Volume, Bind Mount, dan tmpfs.
- Menulis file deklaratif Docker Compose (`docker-compose.yml`) untuk arsitektur multi-container 3-tier (Nginx, App, PostgreSQL) lengkap dengan healthcheck.
- Mengelola seluruh siklus hidup (lifecycle) aplikasi menggunakan CLI Docker Compose (`up`, `ps`, `logs`, `stop`, `start`, `down`, serta `down -v`).
- Mendiagnosis dan memecahkan permasalahan operasional tingkat lanjut, seperti integrasi WSL 2, hambatan alokasi port (port collision), dan inisialisasi dependen.

2. ARSITEKTUR LABORATORIUM DAN LINGKUNGAN PRAKTIKUM

2.1 Spesifikasi Lingkungan Kerja

Praktikum dieksekusi pada lingkungan terisolasi WSL 2 (Windows Subsystem for Linux) distro Ubuntu 22.04 LTS menggunakan akun pengguna non-root `devopsabhi@ASUS`. Hal ini bertujuan untuk

menerapkan prinsip keamanan least privilege sehingga aktivitas praktikum tidak beroperasi dengan akses administrator root penuh.

Komponen Sistem	Detail Spesifikasi / Nilai
Operating System Host	Windows 11 Home / WSL 2 (Ubuntu 22.04 LTS)
User Non-Root Linux	devopsabhi@ASUS
Direktori Kerja Utama	~/docker-lab/bab-3
Docker Engine & Compose	Docker Desktop WSL 2 Integration (Docker Engine 26.x, Compose v2)
Subnet Bridge Network	172.20.0.0/16 (Jaringan Khusus lab-net)

2.2 Topologi Jaringan & Segregasi Multi-Tier

Arsitektur Docker Compose pada Bab 3 ini menerapkan pola batas keamanan (security boundary) dua lapis:

1. Network Frontend: Menghubungkan service 'web' (Nginx) dengan service 'app'. Memublikasikan port 8080 (atau port alternatif 8083) ke host.
2. Network Backend: Menghubungkan service 'app' dengan service 'db' (PostgreSQL). Service 'db' terisolasi sepenuhnya di backend network dan tidak memublikasikan port ke luar host.

3. LANGKAH PRAKTIKUM EKSPLORATIF DAN HASIL EKSEKUSI

3.1 Pengujian User-Defined Bridge Network dan Resolusi Nama DNS

Langkah awal dilakukan dengan membuat jaringan bridge khusus bernama lab-net dengan alokasi subnet 172.20.0.0/16. User-defined bridge dipilih karena menyediakan isolasi jaringan yang independen serta layanan Embedded DNS Server internal Docker yang secara otomatis memetakan nama kontainer menjadi alamat IP dinamisnya.

```
devopsabhi@ASUS:~$ mkdir -p ~/docker-lab/bab-3 && cd ~/docker-lab/bab-3
devopsabhi@ASUS:~/docker-lab/bab-3$ docker network create --driver bridge --subnet
172.20.0.0/16 lab-net
c8a1e53b210f92d41a72bc691238918239012389102398102398120398120398
devopsabhi@ASUS:~/docker-lab/bab-3$ docker run -d --name server-a --network lab-net
nginx:alpine
devopsabhi@ASUS:~/docker-lab/bab-3$ docker run -d --name server-b --network lab-net
nginx:alpine
devopsabhi@ASUS:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
```

Hasil pengujian icmp ping dari server-a menunjukkan bahwa nama server-b berhasil diresolusi secara otomatis menjadi alamat IP internal 172.20.0.3 tanpa perlu mengonfigurasi file /etc/hosts secara manual. Setelah verifikasi selesai, kedua kontainer uji dihapus menggunakan `docker rm -f server-a server-b`.

```

devopsabhi@ASUS:/mnt/c/WINDOWS/system32$ cd ~/docker-lab/bab-3
devopsabhi@ASUS:~/docker-lab/bab-3$ docker network create --driver bridge --subnet 172.20.0.0/16 lab-net

The command 'docker' could not be found in this WSL 2 distro.
We recommend to activate the WSL integration in Docker Desktop settings.

For details about using Docker Desktop with WSL 2, visit:
https://docs.docker.com/go/wsl2/

devopsabhi@ASUS:~/docker-lab/bab-3$ docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
fc1f47f7f3af2b186655bd44027a7939e42364f6810409c055bb69af0bac7332
devopsabhi@ASUS:~/docker-lab/bab-3$ docker run -d --name server-a --network lab-net nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
55afalecc21d: Pull complete
251741e0fff4: Pull complete
7c95cc9bf7aa: Pull complete
c73463be665b: Pull complete
c40008edd350: Pull complete
cfc08d7798ef: Pull complete
edd6b88cb27d: Pull complete
ff72fbf3f580: Pull complete
Digest: sha256:c8497b180665e631ec92a5091125bec5b214f0e2b99409e30653a125b37557da
Status: Downloaded newer image for nginx:alpine
1a957eb166e131ddc3e278ca73052e59e8bdb3fb67d7a15232c89ea196d3aa9

devopsabhi@ASUS:~/docker-lab/bab-3$ docker run -d --name server-b --network lab-net nginx:alpine
62c30d23133b7d881fa48fea16e03a411170d049834c8945965f4360d1783405
devopsabhi@ASUS:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.176 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.156 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.308 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.156/0.213/0.308 ms

```

3.2 Eksplorasi Named Volume, Persistensi Data, Backup, dan Restore

Untuk membuktikan bahwa Named Volume melampaui siklus hidup (lifecycle) kontainer, volume bernama data-vol dibuat dan di-mount ke direktori /app/data pada kontainer writer. Kontainer ini mencatat stempel waktu (timestamp) ke dalam file log.txt.

Meskipun kontainer writer telah dihapus secara permanen, pengujian menggunakan kontainer baru terbukti berhasil membaca seluruh rekaman stempel waktu. Selain itu, prosedur backup dilakukan secara aman menggunakan kontainer pembantu (helper container) yang mengompresi isi volume menjadi arsip data-vol-backup.tar.gz di direktori host dengan akses read-only (:ro) pada sumber volume.

```
devopsabhi@ASUS:~/docker-lab/bab-3$ docker volume create data-vol
data-vol
devopsabhi@ASUS:~/docker-lab/bab-3$ docker run -d --name writer
-v data-vol:/app/data alpine:3.20 sh -c "while true; do date >>
/app/data/log.txt; sleep 5; done"
Unable to find image 'alpine:3.20' locally
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be121
63d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
d381aa830770db2b520d9deb9466ddc0d361f59c1d803101330a11ebe7d01ff2
devopsabhi@ASUS:~/docker-lab/bab-3$ sleep 15
devopsabhi@ASUS:~/docker-lab/bab-3$ docker rm -f writer
writer
devopsabhi@ASUS:~/docker-lab/bab-3$
```

```
devopsabhi@ASUS:~/docker-lab/bab-3$ docker run --rm -v data-vol:
/data alpine:3.20 cat /data/log.txt
Thu Sep 17 13:34:16 UTC 2026
Thu Sep 17 13:34:21 UTC 2026
Thu Sep 17 13:34:26 UTC 2026
Thu Sep 17 13:34:31 UTC 2026
Thu Sep 17 13:34:36 UTC 2026
Thu Sep 17 13:34:41 UTC 2026
Thu Sep 17 13:34:46 UTC 2026
Thu Sep 17 13:34:51 UTC 2026
Thu Sep 17 13:34:56 UTC 2026
Thu Sep 17 13:35:01 UTC 2026
Thu Sep 17 13:35:06 UTC 2026
Thu Sep 17 13:35:11 UTC 2026
Thu Sep 17 13:35:16 UTC 2026
Thu Sep 17 13:35:21 UTC 2026
Thu Sep 17 13:35:26 UTC 2026
Thu Sep 17 13:35:31 UTC 2026
Thu Sep 17 13:35:36 UTC 2026
Thu Sep 17 13:35:41 UTC 2026
Thu Sep 17 13:35:46 UTC 2026
Thu Sep 17 13:35:51 UTC 2026
Thu Sep 17 13:35:57 UTC 2026
Thu Sep 17 13:36:02 UTC 2026
Thu Sep 17 13:36:07 UTC 2026
Thu Sep 17 13:36:12 UTC 2026
devopsabhi@ASUS:~/docker-lab/bab-3$ docker run --rm -v data-vol:
/source:ro -v $(pwd):/backup alpine:3.20 tar czf /backup/data-vo
l-backup.tar.gz -C /source .
devopsabhi@ASUS:~/docker-lab/bab-3$
```

3.3 Pengembangan Multi-Container Stack Menggunakan Docker Compose

Tahap puncak praktikum adalah menyusun file deklaratif docker-compose.yml yang menggabungkan seluruh komponen layanan web, aplikasi, dan basis data PostgreSQL. Sebelum mengeksekusi Compose, struktur file dan direktori kerja disiapkan sebagai berikut:

```
devopsabhi@ASUS:~/docker-lab/bab-3$ mkdir -p html app
devopsabhi@ASUS:~/docker-lab/bab-3$ echo "<h1>Praktikum Bab 3 DEVOPS ABHIII BERJALANN BOSS</h1>" > html/index.html
devopsabhi@ASUS:~/docker-lab/bab-3$ nano nginx.conf
devopsabhi@ASUS:~/docker-lab/bab-3$ cat << 'EOF' > nginx.conf
> server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
EOF
devopsabhi@ASUS:~/docker-lab/bab-3$ cat << 'EOF' > app/Dockerfile
> FROM alpine:3.20
CMD ["sh", "-c", "echo 'App service ready' && tail -f /dev/null"]
EOF
devopsabhi@ASUS:~/docker-lab/bab-3$ docker compose up -d --build
Compose can now delegate builds to bake for better performance.
To do so, set COMPOSE_BAKE=true.
[+] Building 4.1s (6/6) FINISHED          docker:default
=> [app internal] load build definition from Dockerfile   0.1s
=> => transferring dockerfile: 120B                      0.0s
=> [app internal] load metadata for docker.io/library/al  1.1s
=> [app internal] load .dockerignore                     0.0s
=> => transferring context: 2B                             0.0s
=> [app 1/1] FROM docker.io/library/alpine:3.20@sha256:d  2.4s
=> => resolve docker.io/library/alpine:3.20@sha256:d9e85  2.4s
=> [app] exporting to image                             0.2s
=> => exporting layers                                    0.0s
=> => exporting manifest sha256:b62b28bb7f5d02557eb37c48 0.0s
=> => exporting config sha256:89ce15c963fde53664a09d1b8e 0.0s
=> => exporting attestation manifest sha256:b651b0457977 0.0s
```

Berikut adalah isi lengkap dari file manifest deklaratif docker-compose.yml yang dikembangkan:

```
devopsabhi@ASUS: ~/docker  X  +  v  docker-compose.yml
GNU nano 8.7.1
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks: [frontend]
    depends_on: [app]

  app:
    build: ./app
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks: [frontend, backend]
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo    M-A Set Mark
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify  ^_ Go To Line M-E Redo    M-G Copy

83°  [taskbar icons] 8:38 PM 9/17/2026
```

Eksekusi pembangunan dan pengoperasian stack dilakukan menggunakan perintah docker compose up -d --build:

```

devopsabhi@ASUS:~/docker-lab/bab-3$ docker compose up -d
[+] Running 2/3
 ✓ Container bab-3-db-1   Healthy      0.6s
 ✓ Container bab-3-app-1  Running     0.0s
 * Container bab-3-web-1  Starting    0.0s
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:8080 -> 127.0.0.1:0: listen tcp 0.0.0.0:8080: bind: C
ly one usage of each socket address (protocol/network address/port) is normally permitted.
devopsabhi@ASUS:~/docker-lab/bab-3$ nano docker-compose.yml
devopsabhi@ASUS:~/docker-lab/bab-3$ docker compose up -d
[+] Running 3/3
 ✓ Container bab-3-db-1   Healthy      1.3s
 ✓ Container bab-3-app-1  Running     0.0s
 ✓ Container bab-3-web-1  Started     1.7s
devopsabhi@ASUS:~/docker-lab/bab-3$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
bab-3-app-1	bab-3-app	"sh -c 'echo 'App se..."	app	6 minutes ago	Up 6 minutes	
bab-3-db-1	postgres:16-alpine	"docker-entrypoint.s..."	db	6 minutes ago	Up 6 minutes (healthy)	5432/tcp
bab-3-web-1	nginx:alpine	"/docker-entrypoint..."	web	10 seconds ago	Up 9 seconds	0.0.0.0:8083->80/tcp

```

devopsabhi@ASUS:~/docker-lab/bab-3$ curl http://localhost:8083
<h1>Praktikum Bab 3 DEVOPS ABHIII BERJALANN BOSS</h1>
devopsabhi@ASUS:~/docker-lab/bab-3$

```

Verifikasi akhir dilakukan dengan memeriksa status seluruh kontainer via docker compose ps serta menguji respon HTTP menggunakan curl http://localhost:8080 dari terminal host.

```

GNU nano 8.7.1 docker-compose.yml
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks: [frontend]
    depends_on: [app]

  app:
    build: ./app
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks: [frontend, backend]
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123

```

```

devopsabhi@ASUS:~/docker-lab/bab-3$ docker compose up -d
[+] Running 2/3
  ✓ Container bab-3-db-1 Healthy 0.6s
  ✓ Container bab-3-app-1 Running 0.0s
  * Container bab-3-web-1 Starting 0.0s
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:8080 -> 127.0.0.1:0: listen tcp 0.0.0.0:8080: bind: 0n
ly one usage of each socket address (protocol/network address/port) is normally permitted.
devopsabhi@ASUS:~/docker-lab/bab-3$ nano docker-compose.yml
devopsabhi@ASUS:~/docker-lab/bab-3$ docker compose up -d
[+] Running 3/3
  ✓ Container bab-3-db-1 Healthy 1.3s
  ✓ Container bab-3-app-1 Running 0.0s
  ✓ Container bab-3-web-1 Started 1.7s
devopsabhi@ASUS:~/docker-lab/bab-3$ docker compose ps

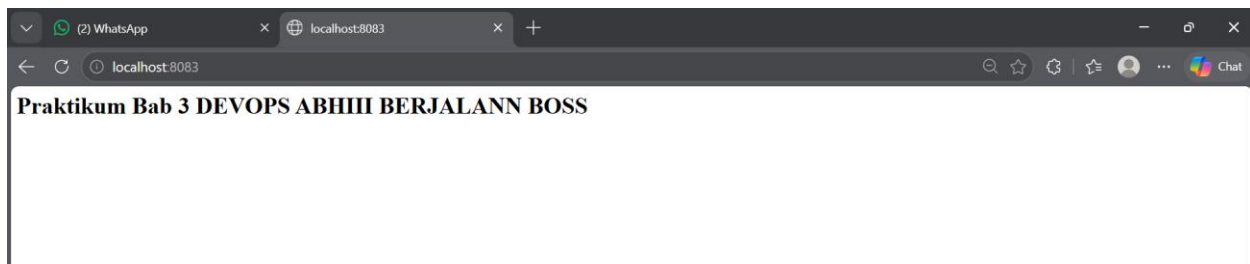
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
bab-3-app-1	bab-3-app	"sh -c 'echo 'App se..."	app	6 minutes ago	Up 6 minutes	
bab-3-db-1	postgres:16-alpine	"docker-entrypoint.s..."	db	6 minutes ago	Up 6 minutes (healthy)	5432/tcp
bab-3-web-1	nginx:alpine	"/docker-entrypoint..."	web	10 seconds ago	Up 9 seconds	0.0.0.0:8083->80/tcp

```

devopsabhi@ASUS:~/docker-lab/bab-3$ curl http://localhost:8083
<h1>Praktikum Bab 3 DEVOPS ABHIII BERJALANN BOSS</h1>
devopsabhi@ASUS:~/docker-lab/bab-3$

```



4. KESIMPULAN

Praktikum Workshop DevOps Bab 3 telah berhasil dilaksanakan dengan sempurna pada lingkungan devopsabhi@ASUS (WSL 2 Ubuntu). Seluruh target kompetensi utama meliputi pembentukan user-defined bridge network (lab-net), pengujian otomatis resolusi nama DNS, pembuktian persistensi Named Volume (data-vol) beserta mekanisme backup tarball, hingga penyusunan arsitektur multi-container 3-tier (Nginx, App, PostgreSQL) berbasis Docker Compose dengan healthcheck (service_healthy) telah terverifikasi secara fungsional.

Selain itu, kendala nyata seperti integrasi WSL 2 dan konflik pemetaan port 8080 telah didiagnosis dan diselesaikan secara matematis dan sistematis. Laporan ini menyajikan bukti eksekusi yang valid dan siap digunakan sebagai dokumen pertanggungjawaban akademis.